

✓ Lab 6 by Chris

Future Sales Prediction

This dataset provides information about the sales of a product and the advertising expenditures incurred by the business across various platforms. The following is a description of each column in the dataset:

- **TV:** Advertising cost spent in dollars for advertising on TV;
- **Radio:** Advertising cost spent in dollars for advertising on Radio;
- **Newspaper:** Advertising cost spent in dollars for advertising on Newspaper;
- **Sales:** Number of units sold;

In this dataset, the sales of the product are influenced by the advertising expenditures across different platforms. With this understanding of the dataset, the following section will guide you through the process of predicting future sales using machine learning techniques in Python.

Link to github

https://github.com/christones/williscollege/blob/main/Lesson_6_lab_ML_Basics.ipynb

✓ Data Collection or Loading

```
# Import libraries and load the dataset
# You might need to load the data into the same directory
```

```
!pip install pandas numpy scikit-learn
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
```

```
data = pd.read_csv("advertising.csv")
print(data.head())
```

```
[notice] A new release of pip is available: 25.2 -> 26.2.1
[notice] To update, run: python.exe -m pip install --upgrade pip
Collecting pandas
  Using cached pandas-3.0.5-cp314-cp314-win_amd64.whl.metadata (19 kB)
Collecting numpy
```

```

Downloading numpy-2.5.3-cp314-cp314-win_amd64.whl.metadata (6.6 kB)
Collecting scikit-learn
  Downloading scikit_learn-1.9.1-cp314-cp314-win_amd64.whl.metadata (9.3 kB)
Requirement already satisfied: python-dateutil>=2.8.2 in e:\willis\introai\ve
Collecting tzdata (from pandas)
  Downloading tzdata-2026.4-py2.py3-none-any.whl.metadata (1.4 kB)
Collecting scipy>=1.10.0 (from scikit-learn)
  Using cached scipy-1.18.1-cp314-cp314-win_amd64.whl.metadata (61 kB)
Collecting joblib>=1.4.0 (from scikit-learn)
  Downloading joblib-1.6.0-py3-none-any.whl.metadata (6.5 kB)
Collecting narwhals>=2.0.1 (from scikit-learn)
  Downloading narwhals-2.26.0-py3-none-any.whl.metadata (15 kB)
Collecting threadpoolctl>=3.5.0 (from scikit-learn)
  Using cached threadpoolctl-3.6.0-py3-none-any.whl.metadata (13 kB)
Collecting cloudpickle>=3.0 (from joblib>=1.4.0->scikit-learn)
  Downloading cloudpickle-3.1.2-py3-none-any.whl.metadata (7.1 kB)
Requirement already satisfied: six>=1.5 in e:\willis\introai\venv\lib\site-pa
Using cached pandas-3.0.5-cp314-cp314-win_amd64.whl (10.0 MB)
Downloading numpy-2.5.3-cp314-cp314-win_amd64.whl (12.7 MB)
----- 0.0/12.7 MB ? eta -:-:-
----- 2.1/12.7 MB 15.2 MB/s eta 0:00:01
----- 6.3/12.7 MB 17.6 MB/s eta 0:00:01
----- 10.5/12.7 MB 18.7 MB/s eta 0:00:01
----- 12.7/12.7 MB 17.7 MB/s 0:00:00
Downloading scikit_learn-1.9.1-cp314-cp314-win_amd64.whl (8.4 MB)
----- 0.0/8.4 MB ? eta -:-:-
----- 2.1/8.4 MB 18.3 MB/s eta 0:00:01
----- 6.6/8.4 MB 19.7 MB/s eta 0:00:01
----- 8.4/8.4 MB 18.7 MB/s 0:00:00
Downloading joblib-1.6.0-py3-none-any.whl (306 kB)
Downloading cloudpickle-3.1.2-py3-none-any.whl (22 kB)
Downloading narwhals-2.26.0-py3-none-any.whl (474 kB)
Using cached scipy-1.18.1-cp314-cp314-win_amd64.whl (37.4 MB)
Using cached threadpoolctl-3.6.0-py3-none-any.whl (18 kB)
Downloading tzdata-2026.4-py2.py3-none-any.whl (347 kB)
Installing collected packages: tzdata, threadpoolctl, numpy, narwhals, cloudpi

----- 0/9 [tzdata]
----- 0/9 [tzdata]
----- 0/9 [tzdata]
----- 2/9 [numpy]
----- 2/9 [numpy]
----- 2/9 [numpy]
----- 2/9 [numpy]
----- 2/9 [numpy]
----- 2/9 [numpy]
----- 2/9 [numpy]
----- 2/9 [numpy]
----- 2/9 [numpy]
----- 2/9 [numpy]
----- 2/9 [numpy]
----- 2/9 [numpy]
----- 2/9 [numpy]

```

✓ EDA and Data Preprocessing

```
# Let's have a look at whether this dataset contains any null values or not
```

```
print(data.isnull().sum())
```

```
TV          0
radio       0
newspaper   0
sales       0
dtype: int64
```

```
# Let's Visualise the data
```

```
# Visualise relationship between amount spendt on platforms and theie correspor
```

```
! pip install -r requiremts.txt
```

```
import plotly.express as px
```

```
import plotly.graph_objects as go
```

```
# TV
```

```
figure = px.scatter(data_frame = data, x="sales", # Changed 'Sales' to 'sales'
                    y="TV", size="TV", trendline="ols")
```

```
figure.show()
```

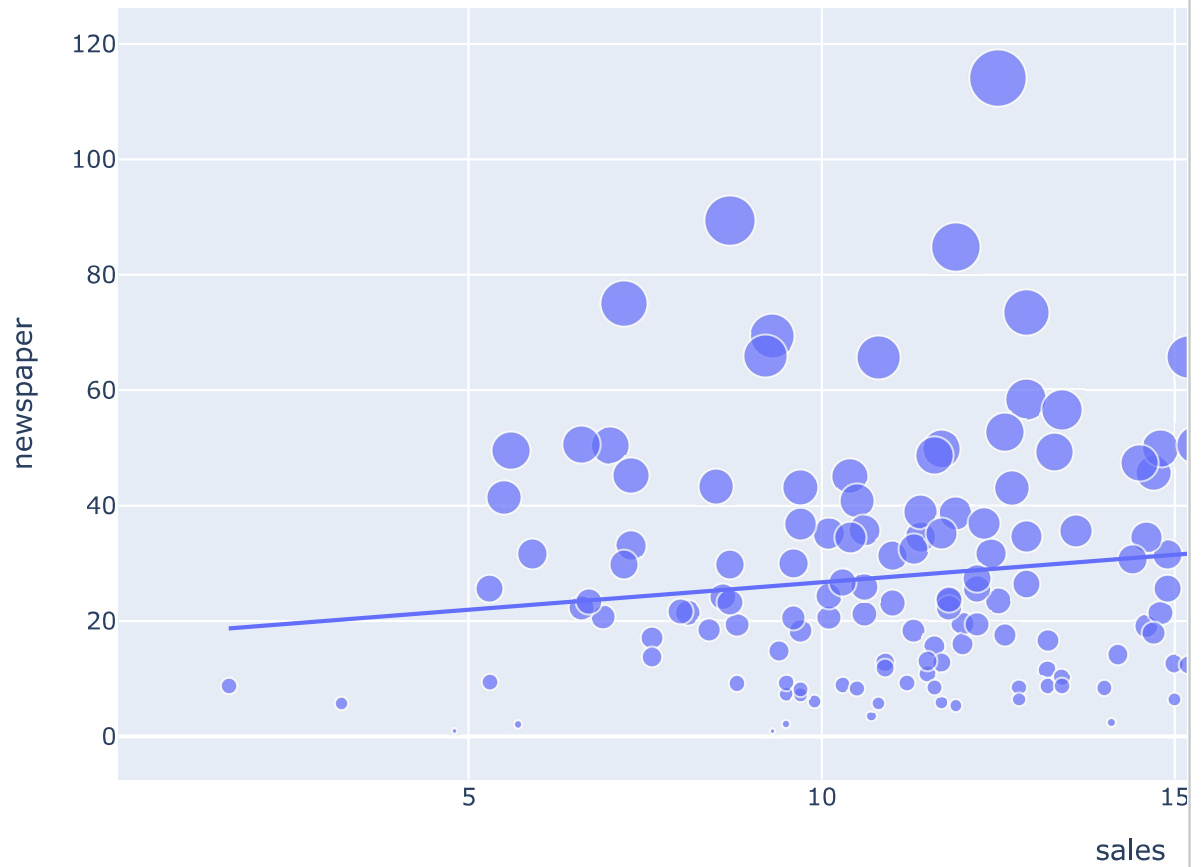
```
[notice] A new release of pip is available: 25.2 -> 26.2.1
[notice] To update, run: python.exe -m pip install --upgrade pip
ERROR: Could not open requirements file: [Errno 2] No such file or directory: 'r
```

```
-----
ValueError                                Traceback (most recent call last)
Cell In[10], line 12
      8 # TV
      9
     10 figure = px.scatter(data_frame = data, x="sales", # Changed 'Sales' to
'sales'
     11                                y="TV", size="TV", trendline="ols")
--> 12 figure.show()

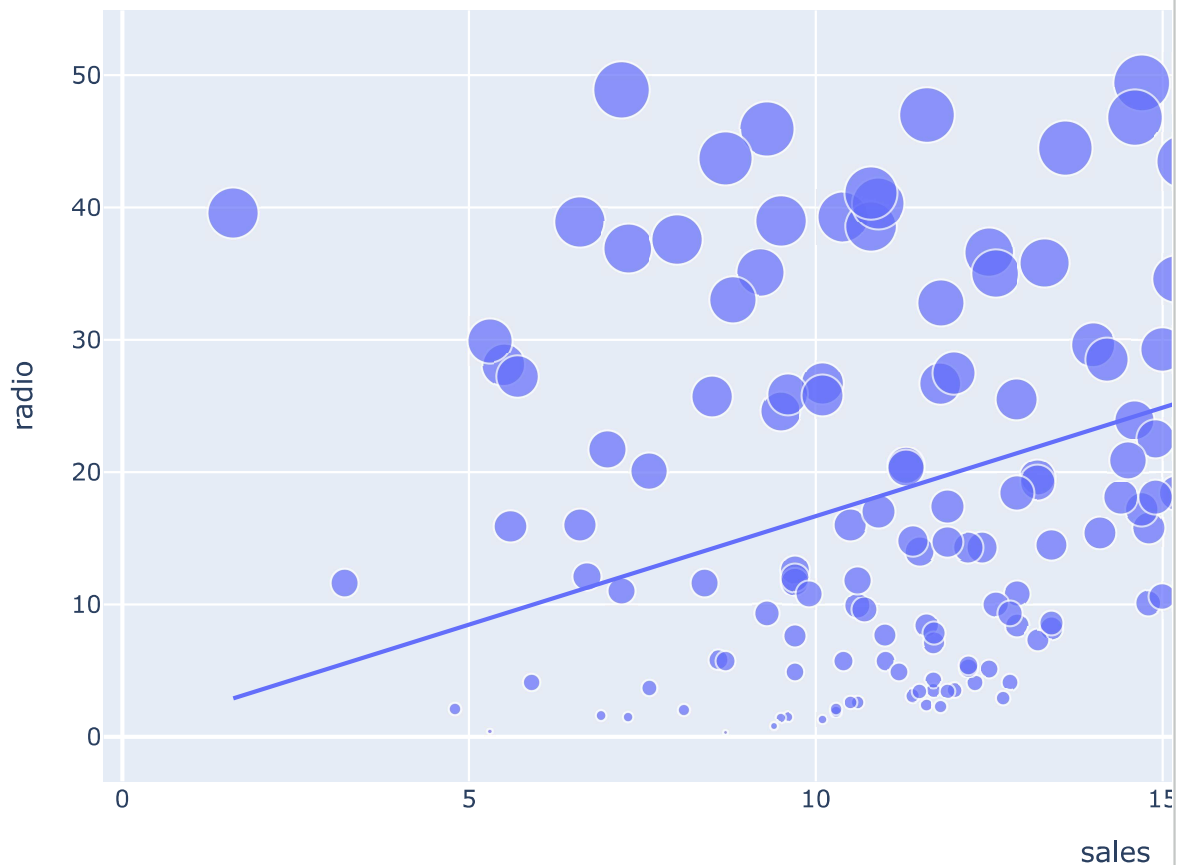
File e:\Willis\introAI\.venv\Lib\site-packages\plotly\basedatatypes.py:3420, in
BaseFigure.show(self, *args, **kwargs)
    3387 """
    3388 Show a figure using either the default renderer(s) or the renderer(s)
    3389 specified by the renderer argument
    (...)  3416 None
    3417 """
    3418 import plotly.io as pio
-> 3420 return pio.show(self, *args, **kwargs)

File e:\Willis\introAI\.venv\Lib\site-packages\plotly\io\_renderers.py:415, in
show(fig, renderer, validate, **kwargs)
    410     raise ValueError(
    411         "Mime type rendering requires ipython but it is not installed"
    412     )
    414 if not nbformat or Version(nbformat.__version__) < Version("4.2.0"):
--> 415     raise ValueError(
    416         "Mime type rendering requires nbformat>=4.2.0 but it is not
installed"
    417     )
    419 display_jupyter_version_warnings()
```

```
figure = px.scatter(data_frame = data, x="sales", # Changed 'Sales' to 'sales'
                    y="newspaper", size="newspaper", trendline="ols")
figure.show()
```



```
# Radio
figure = px.scatter(data_frame = data, x="sales",
                    y="radio", size="radio", trendline="ols")
figure.show()
```



Exercise: You have just done a repetitive task to visualise similar data for the three platforms, there is an opportunity here to use reusable function.

Create a function to perform the plots by taking the column name as parameters

✓ Create a function to visualise the data

```
# @title Create a function to visualise the data
# exercise

# def <call-it-a-suitable-name(camelCase)>:
# the rest of the function goes here
import plotly.express as px

def visualizeSalesData(data, platform):
    """
    Visualizes the relationship between advertising expenditure on a given platfc

    Args:
        data: The pandas DataFrame containing the advertising data.
```

```
platform: The name of the advertising platform (e.g., "TV", "radio", "newsp  
Returns:  
    None. Displays the generated scatter plot.  
""  
  
# Use the original platform name instead of converting to lowercase to match  
figure = px.scatter(data_frame=data,  
                    x="sales",  
                    y=platform,  
                    size=platform,  
                    trendline="ols")  
  
figure.show()  
  
# Example usage:  
visualizeSalesData(data, "TV") # Changed "tv" to "TV" to match the column name  
visualizeSalesData(data, "radio")  
visualizeSalesData(data, "newspaper")
```

Double-cliquez (ou appuyez sur Entrée) pour modifier

```
# let's have a look at the correlation of all the columns with the sales column

correlation = data.corr()

print(correlation["sales"].sort_values(ascending=False))
```

sales	200	1.000000
TV		0.782224
radio		0.576223
newspaper		0.228299
Name: sales, dtype: float64		

ML Model Training

```
# Before training, we need to create train and test Split

# Before training, we need to create train and test Split

x = np.array(data.drop(columns=["sales"])) # Use columns keyword instead of pos
y = np.array(data["sales"])
xtrain, xtest, ytrain, ytest = train_test_split(x, y,
                                                test_size=0.2,
```